

```

print("1. BAYES THEOREM")
print("-" * 50)

# Imagine 100 people take a disease test
total_people = 100

# 10 people actually have the disease
diseased_people = 10

# Among the 10 diseased people, 9 test positive
sick_and_positive = 9

# Among the 90 healthy people, 9 wrongly test positive
healthy_and_positive = 9

# Total positive test results
total_positive = sick_and_positive + healthy_and_positive

# Bayes result:
# Actual sick positive people / All positive people
posterior_probability = sick_and_positive / total_positive

print("Total people:", total_people)
print("People with disease:", diseased_people)
print("Sick people who tested positive:", sick_and_positive)
print("Healthy people who tested positive:", healthy_and_positive)
print("Total positive results:", total_positive)

print(
    "Probability of disease after positive test:",
    posterior_probability * 100,
    "%"
)

```

```

1. BAYES THEOREM
-----
Total people: 100
People with disease: 10
Sick people who tested positive: 9
Healthy people who tested positive: 9
Total positive results: 18
Probability of disease after positive test: 50.0 %

```

```

# =====
# 2. CONCEPT LEARNING USING MAP
# =====

print("\n2. CONCEPT LEARNING USING MAP")
print("-" * 50)

# Training data:
# Study hours, Attendance, Actual result
# 1 = Pass
# 0 = Fail

students = [
    [6, 90, 1],
    [6, 60, 0],
    [3, 90, 0],
    [5, 80, 1],

```

```
[2, 40, 0]
]

# Rule 1:
# Student passes when study hours are at least 5
def rule1(study_hours, attendance):
    return 1 if study_hours >= 5 else 0

# Rule 2:
# Student passes when attendance is at least 75%
def rule2(study_hours, attendance):
    return 1 if attendance >= 75 else 0

# Rule 3:
# Student passes only when both conditions are satisfied
def rule3(study_hours, attendance):
    return 1 if study_hours >= 5 and attendance >= 75 else 0

rules = {
    "Rule 1: Study hours": rule1,
    "Rule 2: Attendance": rule2,
    "Rule 3: Study hours and attendance": rule3
}

# All rules initially have equal prior probability
prior_probability = 1 / 3

best_rule = None
highest_map_score = -1

for rule_name, rule_function in rules.items():

    correct_predictions = 0

    for study_hours, attendance, actual_result in students:

        predicted_result = rule_function(
            study_hours,
            attendance
        )

        if predicted_result == actual_result:
            correct_predictions += 1

    # Likelihood = correct predictions / total examples
    likelihood = correct_predictions / len(students)

    # MAP score = Likelihood × Prior
    map_score = likelihood * prior_probability

    print("\n", rule_name)
    print("Correct predictions:", correct_predictions, "out of", len(students))
    print("Likelihood:", round(likelihood, 2))
    print("Prior:", round(prior_probability, 2))
    print("MAP score:", round(map_score, 4))

    if map_score > highest_map_score:
        highest_map_score = map_score
        best_rule = rule_name
```

```
print("\nBest rule selected by the machine:")
print(best_rule)
```

2. CONCEPT LEARNING USING MAP

```
-----

Rule 1: Study hours
Correct predictions: 4 out of 5
Likelihood: 0.8
Prior: 0.33
MAP score: 0.2667

Rule 2: Attendance
Correct predictions: 4 out of 5
Likelihood: 0.8
Prior: 0.33
MAP score: 0.2667

Rule 3: Study hours and attendance
Correct predictions: 5 out of 5
Likelihood: 1.0
Prior: 0.33
MAP score: 0.3333
```

```
Best rule selected by the machine:
Rule 3: Study hours and attendance
```

```
# =====
# 3. MAXIMUM LIKELIHOOD
# =====

print("\n3. MAXIMUM LIKELIHOOD")
print("-" * 50)

# Actual student marks
actual_marks = [50, 60, 70]

# Predictions from two different models
model1_predictions = [49, 61, 69]
model2_predictions = [30, 80, 90]

# Function to calculate squared prediction error
def squared_error(actual, predicted):

    total_error = 0

    for actual_value, predicted_value in zip(actual, predicted):

        error = actual_value - predicted_value
        total_error += error ** 2

    return total_error

model1_error = squared_error(
    actual_marks,
    model1_predictions
)

model2_error = squared_error(
    actual_marks,
```

```
model2_predictions
)

print("Actual marks:", actual_marks)

print("\nModel 1 predictions:", model1_predictions)
print("Model 1 squared error:", model1_error)

print("\nModel 2 predictions:", model2_predictions)
print("Model 2 squared error:", model2_error)

# Smaller error means greater likelihood
if model1_error < model2_error:
    selected_model = "Model 1"
else:
    selected_model = "Model 2"

print("\nMaximum Likelihood selects:", selected_model)
```

3. MAXIMUM LIKELIHOOD

Actual marks: [50, 60, 70]

Model 1 predictions: [49, 61, 69]

Model 1 squared error: 3

Model 2 predictions: [30, 80, 90]

Model 2 squared error: 1200

Maximum Likelihood selects: Model 1